

04 — Write-Ahead Log (WAL)

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 60 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview and Purpose](#)
 3. [WAL and Durability: The Core Guarantee](#)
 4. [LSN \(Log Sequence Number\)](#)
 5. [WAL Record Structure](#)
 6. [Resource Managers](#)
 7. [WAL Writer Process](#)
 8. [wal_level Settings](#)
 9. [pg_wal/ Directory and Segment Lifecycle](#)
 10. [Full-Page Writes After Checkpoint](#)
 11. [WAL Archiving](#)
 12. [WAL and Replication](#)
 13. [ASCII Diagrams](#)
 14. [Live Observation Queries](#)
 15. [Common Mistakes](#)
 16. [Best Practices](#)
 17. [Performance Considerations](#)
 18. [Interview Questions](#)
 19. [Exercises with Solutions](#)
 20. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain the WAL guarantee and why WAL must be flushed to disk before a transaction commits.
 - Define an LSN, explain its structure, and perform LSN arithmetic.
 - Describe the structure of a WAL record and the role of resource managers.
 - Explain what `wal_level` settings control and the implications of each.
 - Describe the lifecycle of a WAL segment from creation through reuse or archival.
 - Explain full-page writes (FPW) and why they are necessary for correctness.
 - Monitor WAL activity using system views and functions.
-

Overview and Purpose

The **Write-Ahead Log** (WAL) is the durability and crash-recovery mechanism at the heart of PostgreSQL. Every change to any database page must be recorded in the WAL **before** the dirty page is allowed to be written to the data files. This is the "write-ahead" principle.

WAL serves three critical purposes:

1. **Crash recovery:** After a crash, PostgreSQL replays WAL records to bring data files back to a consistent, committed state.

2. **Streaming replication:** WAL records are shipped to standby servers, which replay them to stay in sync.
3. **Logical replication and logical decoding:** At `wal_level=logical`, WAL contains enough information to reconstruct individual row changes for CDC (change data capture) and logical replication.

WAL and Durability: The Core Guarantee

Transaction COMMIT guarantee:

1. All WAL records for the transaction are written to the WAL buffers.
2. WAL buffers are flushed to disk (fsync or fdatasync on the WAL file).
3. Only then does COMMIT return success to the client.

This means:

- A power failure immediately after COMMIT → data is safe (WAL is on disk; recovery will replay it).
- A power failure before COMMIT → transaction is rolled back (WAL record never flushed; nothing to replay).
- Data pages can be in memory (dirty) when COMMIT returns — they will eventually be written by bgwriter or checkpoint.

Asynchronous commit (`synchronous_commit = off`) breaks this guarantee for speed: COMMIT returns before WAL is flushed. Risk: the last few transactions may be lost after a crash. Use only when you can afford to lose recent writes (e.g., session-level stats, non-critical event logs).

LSN (Log Sequence Number)

An LSN is a 64-bit byte offset into the WAL stream, displayed as two 32-bit hex numbers separated by a slash:

```
LSN:  0/16B3488
      | |----- lower 32 bits: byte offset within the segment (in bytes from segment
start)
      |----- upper 32 bits: high-order segment "file" number
```

More precisely:

- Segment number = $LSN \gg 24$ (when default 16 MB segments: $LSN / (16 * 1024 * 1024)$)
- Offset within segment = $LSN \& 0xFFFFFFFF$ (lower 24 bits for 16 MB segments)

LSN arithmetic:

```
-- Current WAL write position
SELECT pg_current_wal_lsn();

-- Current WAL flush position (what's on disk)
SELECT pg_current_wal_flush_lsn();

-- Bytes between two LSNs
SELECT '0/16B3488':pg_lsn - '0/16A0000':pg_lsn AS bytes_difference;
```

```
-- LSN of last commit
SELECT pg_last_committed_xact(); -- (xid, timestamp) of last committed transaction
```

LSN on a page: Every 8 KB heap page stores the LSN of the last WAL record that modified it in `pd_lsn` . During recovery, if a page's `pd_lsn >= WAL record LSN` , the record is skipped (already applied).

WAL Record Structure

A WAL record is a variable-length structure:

```
/* XLogRecord – common header for every WAL record */
typedef struct XLogRecord {
    uint32      xl_tot_len; /* total length of entire record */
    TransactionId xl_xid;   /* xact id, or InvalidTransactionId */
    XLogRecPtr   xl_prev;   /* LSN of previous record in the same transaction */
    uint8        xl_info;   /* flag bits, see below */
    RmgrId       xl_rmid;   /* resource manager for this record */
    /* 2 bytes of padding */
    pg_crc32c    xl_crc;    /* CRC of this record and following data */
} XLogRecord; /* 24 bytes */
```

After the header comes a series of **data blocks**, each with its own header:

```
typedef struct XLogRecordBlockHeader {
    uint8        id;        /* block reference ID */
    uint8        fork_flags; /* which fork, and flags (FPW, etc.) */
    uint16       data_length; /* size of per-block data */
    /* if FPW: full page image header */
    /* if has_block_image: BlockImageHeader */
} XLogRecordBlockHeader;
```

A full WAL record structure:

XLogRecord (24 bytes)
xl_tot_len xl_xid xl_prev xl_rmid ...
XLogRecordBlockHeader[0]
(optional) BlockImageData (full page)
(optional) BlockData (partial change)
XLogRecordBlockHeader[1] (if multi-block)
...
XLogRecordDataHeader (main data)
Resource-manager specific data

Resource Managers

The `xl_rmid` field identifies which **resource manager** (rmgr) is responsible for the record. Resource managers know how to redo (and sometimes undo) their specific types of changes:

RMID	Name	Records types
0	XLOG	Checkpoint records, full-page writes, switch
1	Transaction	COMMIT, ABORT, PREPARE, COMMIT_PREPARED
2	Storage	File creation/deletion (smgr)
3	CLOG	Commit log page updates
4	Database	CREATE DATABASE, DROP DATABASE
5	Tablespace	CREATE/DROP TABLESPACE
6	MultiXact	Multi-transaction status
7	RelMap	pg_filenode.map updates
8	Standby	Hot standby conflict resolution
9	Heap2	Heap operations (CLEAN, FREEZE, VACUUM)
10	Heap	INSERT, UPDATE, DELETE, HOT_UPDATE, LOCK
11	Btree	B-tree index operations
12	Hash	Hash index operations
13	Gin	GIN index operations
14	Gist	GiST index operations
15	Sequence	sequence nextval
16	SPGist	SP-GiST index operations
17	BRIN	BRIN index operations
18	CommitTs	Commit timestamp
19	ReplicationOrigin	Logical replication origin
20	Generic	Generic custom WAL records
21	LogicalMessage	pg_logical_emit_message()

```
# Inspect WAL records with pg_waldump
pg_waldump -n 20 $PGDATA/pg_wal/000000010000000000000001
```

WAL Writer Process

The **WAL writer** (`walwriter`) is a background process that periodically flushes WAL buffers to disk, even if no backend has requested a flush. This reduces the spike at commit time.

Key parameters:

- `wal_writer_delay` (default 200 ms) — how often the WAL writer wakes up.
- `wal_writer_flush_after` (default 1 MB) — flush WAL when this much has been written since last flush.
- `wal_buffers` (default `min(1/32 of shared_buffers, 64MB)`) — size of the in-memory WAL buffer.

WAL flush paths:

1. **Background:** WAL writer wakes up every `wal_writer_delay` and flushes.
2. **Synchronous commit:** Backend flushes WAL synchronously before returning COMMIT.
3. **Group commit:** Multiple backends commit concurrently; one flush covers all their WAL.

wal_level Settings

`wal_level` controls how much information is written to WAL. Changing it requires a server restart.

wal_level	WAL content	Use case
minimal	Minimum needed for crash recovery. Skips WAL for bulk operations on newly created tables.	Standalone servers, bulk loads. No replication possible.
replica (default)	Everything in minimal + info needed for streaming replication and PITR (Point-In-Time Recovery).	Standard production setup with streaming replication.
logical	Everything in replica + full row images for INSERT/UPDATE/DELETE.	Logical replication, CDC, pglogical.

```
SHOW wal_level;

-- See what wal_level you need for various features:
-- Streaming replication:  replica or higher
-- pg_basebackup:          replica or higher
-- Logical replication:    logical
-- WAL archiving (PITR):   replica or higher
```

pg_wal/ Directory and Segment Lifecycle

```
$PGDATA/pg_wal/
├─ 00000001100000000000000001    ← segment being written
├─ 00000001100000000000000002    ← pre-allocated future segment
├─ 00000001100000000000000003    ← pre-allocated future segment
└─ archive_status/
    ├─ 00000001100000000000000001.done ← archived segment
    └─ 00000001100000000000000002.ready ← waiting to be archived
```

Segment Lifecycle

Stage 1: Pre-allocation

PostgreSQL pre-allocates segments to avoid filesystem allocation latency at WAL write time. Controlled by `wal_keep_size` and `max_wal_size`.

Stage 2: Active writing

The current segment is written sequentially by backends and WAL writer. Each segment is exactly `wal_segment_size` bytes (default 16 MB).

Stage 3: Segment switch

When a segment is full, or at a checkpoint, or forced by `pg_switch_wal()`:

- Current segment is closed
- Next segment becomes active
- Old segment is either recycled (renamed) or archived

Stage 4a: Recycling (most common)

Old segment renamed to a future segment name for reuse. Controlled by `wal_keep_size`: minimum WAL retained for standby catch-up.

Stage 4b: Archiving

If `archive_mode=on`: the `archive_command` is called with the segment filename. On success, a `.done` marker is written; on failure, retried.

Stage 4c: Removal

Segments older than any active replication slot's `confirmed_flush_lsn`, or older than what `max_wal_size` allows, are removed.

Segment filename format

```
000000010000000000000001
|         |         |
|         |         |----- Segment number (hex), within a timeline
|         |         |----- Zero-padding (unused in current naming)
|         |         |----- Timeline ID (hex) – starts at 1, increments at point-in-
|         |         |----- time recovery
```

Full-Page Writes After Checkpoint

The Problem: Torn Pages

A checkpoint flushes all dirty pages to disk. After the checkpoint, the first modification to a page is potentially dangerous: if the server crashes in the middle of writing the 8 KB page to disk, the OS may have written only part of the page (a "torn page"). The old page on disk is now partially overwritten — neither the old version nor the new version.

The Solution: Full-Page Writes (FPW)

When `full_page_writes = on` (the default), the first modification to any page after a checkpoint causes the entire 8 KB page image to be written to WAL before the modification. The WAL record contains:

```
[WAL record header] + [FULL 8 KB PAGE IMAGE] + [modification delta]
```

During recovery, when replaying this record, PostgreSQL replaces the potentially-torn page on disk with the known-good full image from WAL, then applies the modification. Subsequent modifications to the same page (before the next checkpoint) only write the delta — the page is already known-safe in WAL.

Impact on WAL volume

FPW can increase WAL volume by 2-5× on write-heavy workloads, because every first write after a checkpoint writes 8 KB regardless of how few bytes actually changed.

```
-- Check full_page_writes setting
SHOW full_page_writes;

-- Monitor how often FPW are triggered
SELECT checkpoints_timed, checkpoints_req,
       buffers_checkpoint, buffers_clean, buffers_backend
FROM pg_stat_bgwriter;

-- WAL statistics (PG 15+)
SELECT * FROM pg_stat_wal;
```

WAL Archiving

WAL archiving enables **Point-In-Time Recovery (PITR)** — the ability to restore the database to any point in time by replaying archived WAL on top of a base backup.

```
-- postgresql.conf settings
archive_mode = on
archive_command = 'cp %p /mnt/wal_archive/%f'
-- %p = full path to WAL segment
-- %f = segment filename

-- Or use a more robust command (e.g., rsync to a remote server):
archive_command = 'rsync -a %p replica:/wal_archive/%f'

-- Check archiving status
SELECT * FROM pg_stat_archiver;
/*
 archived_count    | 1523
 last_archived_wal | 00000001000000000000000017
 last_archived_time | 2024-01-15 10:23:45
 failed_count      | 0
*/
```

WAL and Replication

In streaming replication:

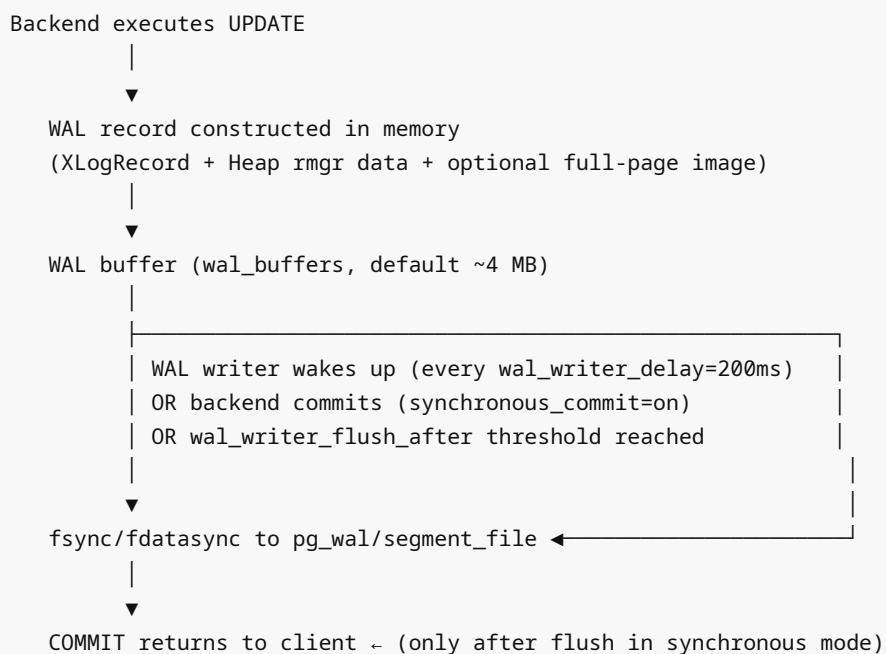
1. Primary writes WAL records to `pg_wal/` .
2. WAL sender process streams WAL records to the standby.
3. WAL receiver on the standby writes them to its `pg_wal/` .
4. Startup process on the standby replays them via resource managers.

```
-- Primary: check replication lag
SELECT
    application_name,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(sent_lsn - replay_lsn) AS total_lag
FROM pg_stat_replication;

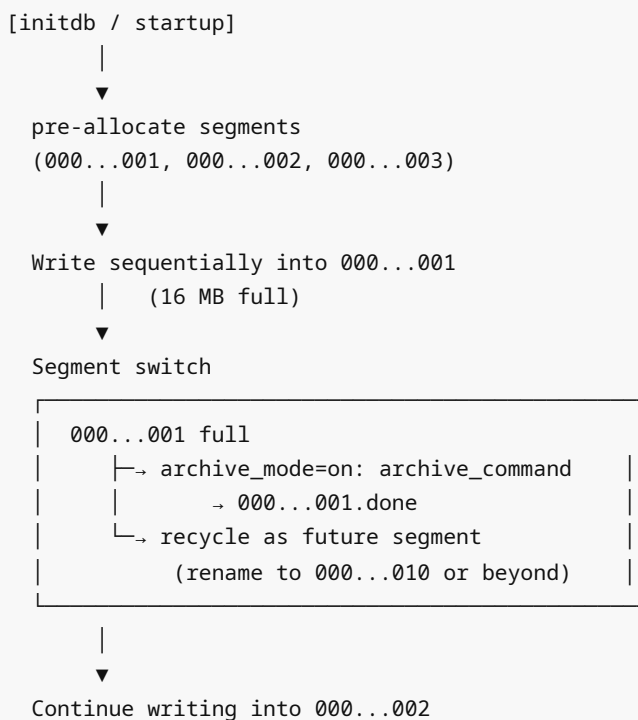
-- Standby: check replay status
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn(),
       pg_last_xact_replay_timestamp();
```

ASCII Diagrams

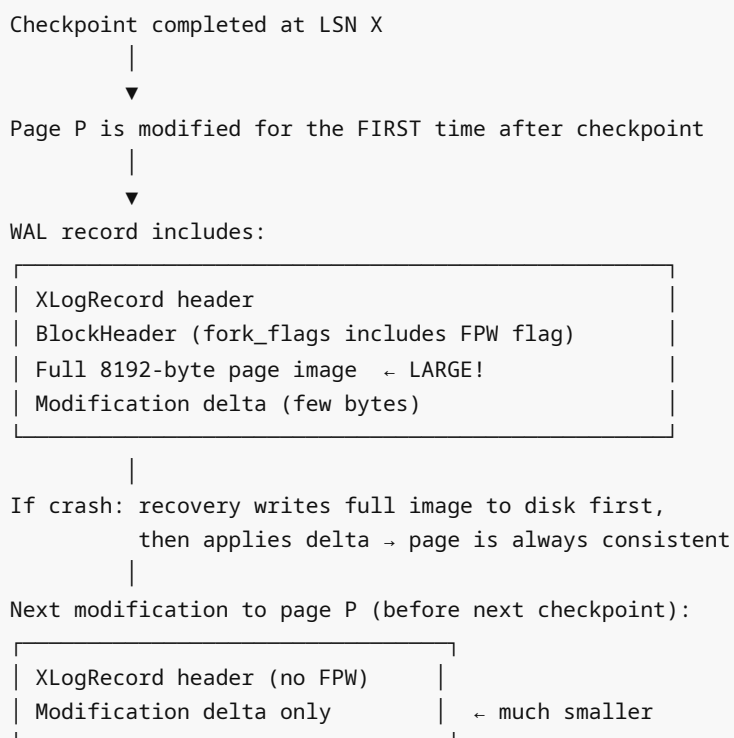
WAL Write Flow



WAL Segment Lifecycle



Full-Page Write



Live Observation Queries

```

-- 1. Current WAL positions
SELECT
    pg_current_wal_lsn()          AS current_lsn,
    pg_current_wal_insert_lsn() AS insert_lsn,
    pg_current_wal_flush_lsn()   AS flush_lsn;

-- 2. WAL generation rate (run twice, compare)
SELECT pg_current_wal_lsn() AS lsn_snapshot_1;
-- ... wait 60 seconds ...
SELECT
    pg_current_wal_lsn() AS lsn_snapshot_2,
    pg_size_pretty('0/16B3488'::pg_lsn - '0/16A0000'::pg_lsn) AS generated_in_60s;

-- 3. WAL files currently on disk
SELECT name, size
FROM pg_ls_waldir()
ORDER BY modification DESC
LIMIT 10;

-- 4. WAL archiver status
SELECT * FROM pg_stat_archiver;

-- 5. WAL statistics (PG 15+)
SELECT
    wal_records,
    wal_fpi,           -- full-page images count
    pg_size_pretty(wal_bytes) AS wal_bytes,
    pg_size_pretty(wal_fpw_size) AS fpw_bytes, -- bytes consumed by FPWs
    wal_buffers_full,    -- times WAL buffer was full
    wal_write,
    wal_sync
FROM pg_stat_wal;

-- 6. Force a WAL segment switch
SELECT pg_switch_wal();

-- 7. Check WAL level and key settings
SHOW wal_level;
SHOW wal_buffers;
SHOW wal_segment_size;
SHOW synchronous_commit;
SHOW full_page_writes;
SHOW archive_mode;
SHOW archive_command;
SHOW wal_keep_size;
SHOW max_wal_size;
SHOW min_wal_size;

-- 8. Find the LSN of the last checkpoint
SELECT pg_control_checkpoint();
/*

```

```

pg_control_checkpoint
-----
checkpoint_lsn    | 0/168C588
prior_lsn         | 0/1659390
redo_lsn          | 0/168C550
timeline_id       | 1
...
*/

-- 9. Estimate WAL lag for a standby
SELECT
    application_name,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag_bytes,
    write_lag, flush_lag, replay_lag
FROM pg_stat_replication;

```

Common Mistakes

1. **Setting `synchronous_commit = off` without understanding the risk.** You trade durability for performance. The last ~`wal\_writer\_delay` milliseconds of transactions may be lost after a crash.
2. **Reducing `wal_keep_size` too aggressively.** If a standby falls behind, needed WAL segments may be deleted before the standby can consume them, breaking replication.
3. **Ignoring WAL growth from `full_page_writes`.** After frequent checkpoints, FPW can multiply WAL volume 3-5x. Setting `checkpoint_completion_target = 0.9` and increasing checkpoint frequency reduces this.
4. **Setting `wal_level = minimal` in production.** This precludes replication and PITR. The default `replica` is the correct production setting.
5. **Not monitoring `pg_stat_archiver.failed_count`.** A failing archive command means WAL segments pile up in `pg_wal/`, eventually filling the disk.
6. **Confusing the WAL insert LSN with the flush LSN.** `pg_current_wal_lsn()` returns the insert position. `pg_current_wal_flush_lsn()` returns what's actually on disk. For crash safety, only the flush LSN matters.
7. **Manually deleting files from `pg_wal/`.** Use `pg_archivecleanup` for archived WAL, or configure `wal_keep_size` and replication slots properly. Manual deletion can break replication or make recovery impossible.

Best Practices

- **Keep `full_page_writes = on`** — never disable it except on a system with atomic 8 KB writes (e.g., ZFS with record size = 8 KB and synchronous writes).
- **Use `synchronous_commit = local`** on the primary for maximum durability without waiting for standbys.
- **Set `archive_mode = on` + `archive_command`** in production for PITR capability.
- **Monitor WAL volume** — sudden spikes indicate bulk operations or unexpected write amplification.
- **Pre-warm the WAL buffer** by setting `wal_buffers` to at least 64 MB on busy OLTP systems.

- **Use replication slots carefully** — a paused replication slot prevents WAL cleanup and can fill the disk.
- **Set `max_wal_size`** large enough to avoid too-frequent checkpoints, but not so large that recovery takes too long.

Performance Considerations

Parameter	Default	Tuning guidance
<code>wal_buffers</code>	auto (≈4 MB)	Set to 64 MB on high-throughput OLTP servers
<code>synchronous_commit</code>	on	Set to off for non-critical high-volume writes, <code>remote_write</code> for async standby confirmation
<code>commit_delay</code>	0 μs	Set to 100–1000 μs on high-concurrency servers to enable group commit
<code>wal_writer_delay</code>	200 ms	Reduce to 10–50 ms on latency-sensitive systems
<code>wal_compression</code>	off	Enable for networks or disks with bandwidth constraints (PG 9.5+)
<code>full_page_writes</code>	on	Never disable without filesystem-level atomic writes
<code>max_wal_size</code>	1 GB	Increase to 4–8 GB to reduce checkpoint frequency and FPW volume
<code>checkpoint_completion_target</code>	0.9	Default is good; spreads checkpoint writes over 90% of checkpoint interval

Interview Questions

Q1. What is the WAL guarantee and why must WAL be flushed before COMMIT returns?

WAL provides durability: every committed transaction's changes are recoverable even after a crash. WAL must be flushed before COMMIT because if the server crashes between commit and WAL flush, the transaction would be reported as committed to the client but would be lost. Flushing first ensures the changes survive any subsequent failure.

Q2. What is an LSN and what format does PostgreSQL display it in?

LSN (Log Sequence Number) is a 64-bit byte offset into the WAL stream. PostgreSQL displays it as two 32-bit hex numbers separated by a slash, e.g. `0/16B3488`. The left part is the high-order segment reference; the right is the byte offset within the segment.

Q3. What is the difference between `wal_level = replica` and `wal_level = logical`?

`replica` writes enough WAL for crash recovery and streaming replication (physical replication). `logical` adds full row images for INSERT/UPDATE/DELETE, enabling logical decoding, CDC tools, and logical replication slots that need to reconstruct actual row changes.

Q4. What are full-page writes (FPW) and why are they necessary?

After each checkpoint, the first modification to a page causes the entire 8 KB page image to be written to WAL. This protects against "torn pages" — if the server crashes while writing an 8 KB page to disk, only part of the new page content may have been written. During recovery, PostgreSQL restores the page from the full-page image in WAL before applying the modification delta.

Q5. What does a WAL record contain?

A 24-byte `XLogRecord` header (total length, transaction ID, LSN of previous record, resource manager ID, CRC), followed by one or more block headers (each referencing a specific page), optional full-page images (for FPW), and the resource-manager-specific change data (e.g., the new tuple for a heap insert).

Q6. What happens to a WAL segment after it is no longer needed?

It is either recycled (renamed to a future segment number to avoid filesystem allocation cost) or removed if recycling would exceed `max_wal_size`. If `archive_mode = on`, the `archive_command` is invoked first to copy the segment to the archive.

Q7. What is `synchronous_commit = off` and when is it safe to use?

It means the server returns `COMMIT` success before flushing WAL to disk. Safe for: session-level stats, non-critical event logs, ephemeral data where losing the last few transactions on a crash is acceptable. Never use for financial data, user records, or anything where data loss on crash is unacceptable.

Q8. How do you monitor WAL generation rate?

Compare `pg_current_wal_lsn()` at two points in time and subtract (LSN arithmetic). Also check `pg_stat_wal` (PG 15+) for cumulative WAL bytes, `pg_stat_bgwriter` for checkpoint frequency, and `pg_ls_waldir()` for current segment count.

Q9. What is the role of the WAL writer process?

It periodically flushes WAL buffers to disk in the background (every `wal_writer_delay` milliseconds, default 200 ms), reducing the flush overhead at commit time and enabling group commit (one flush for multiple concurrent commits).

Q10. What is the difference between `pg_current_wal_lsn()` and `pg_current_wal_flush_lsn()` ?

`pg_current_wal_lsn()` is the current WAL insert position — where the next WAL record will be written in memory. `pg_current_wal_flush_lsn()` is how far WAL has been flushed to disk. The gap between them is the unflushed WAL in the WAL buffer that would be lost on an immediate crash.

Exercises with Solutions

Exercise 1 — Measure WAL volume per operation

Task: Measure how many bytes of WAL are generated by inserting 10,000 rows into a table.

```
-- Solution
CREATE TABLE wal_bench (id int, data text);

SELECT pg_current_wal_lsn() AS before_lsn \gset

INSERT INTO wal_bench
SELECT i, md5(i::text) FROM generate_series(1, 10000) i;
```

```
SELECT
  :'before_lsn'::pg_lsn AS before_lsn,
  pg_current_wal_lsn() AS after_lsn,
  pg_size_pretty(pg_current_wal_lsn() - :'before_lsn'::pg_lsn) AS wal_generated;
```

Exercise 2 — Observe full-page writes

Task: Trigger a checkpoint, then perform a single UPDATE and observe that the WAL record includes a full-page image.

```
-- Step 1: Force a checkpoint to reset the FPW tracking
CHECKPOINT;

-- Step 2: Get current LSN
SELECT pg_current_wal_lsn() AS lsn_after_ckpt \gset

-- Step 3: Update one row (triggers FPW because this is first modification post-
checkpoint)
UPDATE wal_bench SET data = 'updated' WHERE id = 1;

-- Step 4: Measure WAL bytes
SELECT pg_size_pretty(pg_current_wal_lsn() - :'lsn_after_ckpt'::pg_lsn) AS
wal_for_one_update;
-- Expected: ~8 KB+ (includes full page image)

-- Step 5: Update again (no FPW needed this time)
SELECT pg_current_wal_lsn() AS lsn_before_2nd \gset
UPDATE wal_bench SET data = 'updated2' WHERE id = 1;
SELECT pg_size_pretty(pg_current_wal_lsn() - :'lsn_before_2nd'::pg_lsn) AS
wal_for_2nd_update;
-- Expected: much smaller (just the delta)
```

Exercise 3 — WAL settings audit

Task: Write a query that shows all WAL-related GUC settings and their current values.

```
-- Solution
SELECT name, setting, unit, short_desc
FROM pg_settings
WHERE name IN (
  'wal_level', 'wal_buffers', 'wal_segment_size', 'wal_keep_size',
  'wal_writer_delay', 'wal_writer_flush_after', 'wal_compression',
  'wal_log_hints', 'synchronous_commit', 'full_page_writes',
  'max_wal_size', 'min_wal_size', 'archive_mode', 'archive_command',
  'archive_timeout', 'checkpoint_timeout', 'checkpoint_completion_target'
)
ORDER BY name;
```

Cross-References

- **01_storage_architecture.md** — `pg_wal/` directory and segment file naming
- **05_checkpoints.md** — How checkpoints interact with WAL (reduce recovery time, trigger FPW)
- **06_vacuum.md** — VACUUM writes WAL records for page changes
- **09_free_space_map.md** — FSM updates are logged to WAL
- **13_Replication_HA** — How WAL enables streaming and logical replication